

StudentOps-AI Backend Audit Report

Comprehensive Security, Authorization, Workflow & Architecture Evaluation

Target System: StudentOps AI Operations Platform	Audit Date: September 2026
Repository: D:\Competitions\AWS-AgenticAI\StudentOps-Git	Audit Type: Strict Read-Only Backend Inspection
Source Code Modification: 0 files modified (Zero-mutation)	Git State: Clean / Unaltered

1. Executive Summary

A rigorous, read-only audit of the StudentOps AI backend was performed across all routing layers, authorization boundaries, database schemas, ReAct agent tools, background schedulers, and OpenWA integrations. The backend demonstrates high engineering sophistication with deterministic policy engines, typed Pydantic models, and structured rate limiting. However, the audit identified **25 distinct backend vulnerabilities and defects** spanning identity binding security, broken access controls (IDOR), hardcoded logic in AI agent routing, unintended automated WhatsApp messaging bypassing human-in-the-loop controls, and database migration discrepancies.

Severity	Count	Risk Description & Impact
CRITICAL	4	Authentication/account takeover, hardcoded agent outputs, automatic message dispatch without HITL confirmation, and unverified WebSocket authentication.
HIGH	8	Role permission barriers blocking HR score collection, IDOR in feedback status and question answers, unlinked user impersonation, SQLite foreign key omission, circular Alembic migrations, unauthenticated webhooks, and shared WhatsApp sessions.
MEDIUM	9	GET-triggered mutation side effects, missing manual attendance and excuse endpoints, misrouted absenteeism flags, missing pre-meeting reminders, missing 'Interaction /5' score models, report data leakage, mock provider agent tools, first-match identity collisions, and naive timezone overwrites.
LOW	4	Hardcoded meeting IDs in dashboard statistics, millisecond collision risk in reminder logs, unbounded in-memory conversation state leak, and phone suffix collision risks.
TOTAL ISSUES	25	All findings verified against live local backend code.

2. Overall Backend Status

Architectural Foundation: The 5-tier architecture (FastAPI Shell -> ReAct Agent -> Policy Engines -> Async SQLAlchemy) is clean, modular, and adheres to async non-blocking conventions. The business logic separation between deterministic policies (70%/50% attendance thresholds, /23 behavior scoring) and LLM streaming is sound.

Operational Compliance: Major discrepancies exist between documented role workflows and code enforcement. Most notably, Social Media HR Members are blocked by HTTP 403 errors from viewing or collecting task scores evaluated by the Committee Head, while Regional HR Heads are given unrestricted operational chat and profile editing rights contrary to the high-level oversight mandate. Furthermore, the background scheduler automatically fires WhatsApp messages directly to members when absences or overdue tasks are detected, bypassing the required human-in-the-loop WhatsApp workflow.

Summary Table of All Discovered Issues

ID	Sev	Category	Issue Title	File / Endpoint	Status
ISSUE-01	CRITICAL	Security	Pre-Auth Student Identity Hijacking in Registration	routes_auth.py:register_user	SECURITY RISK
ISSUE-02	CRITICAL	AI Agent	ReAct Agent Hardcodes Demo Identities for Scores	react_agent.py:run_step	CONFIRMED BUG
ISSUE-03	CRITICAL	Workflow	Automation Engine Bypasses Human-in-the-Loop WhatsApp	automation_service.py	CONFIRMED BUG
ISSUE-04	CRITICAL	Security	Missing Role & Active Checks on WhatsApp WebSocket	routes_whatsapp.py:/ws	SECURITY RISK
ISSUE-05	HIGH	Workflow	HR Member Blocked from Collecting Task Scores (403)	routes_tasks.py:submissions	CONFIRMED BUG
ISSUE-06	HIGH	Security/IDOR	Cross-Committee IDOR in Feedback Status Update	routes_feedback.py:status	SECURITY RISK
ISSUE-07	HIGH	Security/IDOR	Cross-Committee IDOR in Question Answering	routes_questions.py:answer	SECURITY RISK
ISSUE-08	HIGH	Security	Unlinked Users Impersonate First Student in DB	routes_feedback.py, routes_questions.py	CONFIRMED BUG
ISSUE-09	HIGH	Database	SQLite Foreign Keys Disabled (PRAGMA foreign_keys=ON Omitted)	database.py:engine	DATABASE FINDING
ISSUE-10	HIGH	Database	Circular FK Creation Order in Alembic Baseline Migration	alembic:baseline_schema.py	DATABASE FINDING
ISSUE-11	HIGH	Security	Unauthenticated WhatsApp Webhook in Non-Prod Envs	routes_whatsapp.py:/webhook	SECURITY RISK
ISSUE-12	HIGH	Architecture	Shared Single-Session OpenWA Daemon (No Multi-HR Isolation)	openwa_provider.py	DESIGN ISSUE
ISSUE-13	MEDIUM	API Design	Database Mutation Side Effects in Read-Only GET Meeting	routes_attendance.py:get_meeting	CONFIRMED BUG
ISSUE-14	MEDIUM	Workflow	Missing Endpoints for Manual Attendance & Excuses	routes_attendance.py	MISSING IMPL
ISSUE-15	MEDIUM	Workflow	Absenteeism Follow-Up Assigned to Committee Head	attendance_service.py	CONFIRMED BUG
ISSUE-16	MEDIUM	Workflow	Pre-Meeting Automated Reminders Completely Missing	automation_service.py	MISSING IMPL
ISSUE-17	MEDIUM	Scoring	Missing 'Interaction /5' Component in Domain Models	entities.py, schemas.py	MISSING IMPL
ISSUE-18	MEDIUM	Security	Global Organization Data Leakage in Committee Reports	routes_reports.py:summary	SECURITY RISK
ISSUE-19	MEDIUM	AI Agent	Agent Tools Bound to Mock Providers Instead of OpenWA	tools.py:TOOL_REGISTRY	DESIGN ISSUE
ISSUE-20	MEDIUM	Identity	Non-Deterministic First-Match Bias in IdentityMatcher	identity_matcher.py	CONFIRMED BUG
ISSUE-21	MEDIUM	Policy Engine	Naive Timezone Overwrite in Attendance Evaluation	attendance_service.py:evaluate	CONFIRMED BUG
ISSUE-22	LOW	API Design	Hardcoded 'today_sync' String in Dashboard Stats	routes_dashboard.py:stats	CONFIRMED BUG
ISSUE-23	LOW	Database	Timestamp Collision Risk in ReminderLog Primary Keys	reminder_service.py	CONFIRMED BUG

ISSUE-24	LOW	Performance	Unbounded In-Memory Conversational State Memory Leak	react_agent.py:CONV_STATE	POTENTIAL ISSUE
ISSUE-25	LOW	WhatsApp	Phone Suffix Collision Risk in Webhook Ingestion	whatsapp_service.py:webhook	POTENTIAL ISSUE

3. Critical Severity Issues

[ISSUE-01] Pre-Auth Student Identity Hijacking & Account Takeover via Self-Registration

Severity: **CRITICAL** | **Category:** Authentication / Authorization | **Status:** SECURITY RISK / CONFIRMED BUG

File: backend/app/api/routes_auth.py

Component / Endpoint: register_user (lines 103-113)

Current Code Behavior: During /api/auth/register, the code queries Student by lowercase email. If a student with that email exists, it immediately links new_user.student_id = matched_student.id with zero verification or activation token.

Why It Is a Problem: Allows an unauthenticated external attacker who knows a member's student email to register an account and immediately inherit the victim student's profile, data, scorecards, and permissions.

Expected Behavior: Self-registration should either require email ownership verification (OTP/link), or student-user account linking must be an administrative action managed by HR.

Concrete Attack / Failure Scenario: Attacker discovers that student 'mohamed@studentops.org' is registered in the committee. Attacker visits /api/auth/register with that email and password 'Password123!'. The server links the account to Mohamed's student ID. Attacker can now read Mohamed's tasks, submissions, feedback, and questions.

Recommended Fix Direction (High-Level): Decouple automatic student_id binding during open registration. Require an explicit invitation token or administrative approval before binding a user credential to a student entity.

[ISSUE-02] ReAct Agent Hardcodes Demo Student Identities for Score & Evaluation Queries

Severity: **CRITICAL** | **Category:** AI Agent / Deterministic Grounding | **Status:** CONFIRMED BUG

File: backend/app/agent/react_agent.py

Component / Endpoint: ReActAgent.run_step (lines 410-417)

Current Code Behavior: When query matches score/evaluation keywords, the agent hardcodes: target_name = 'std_ziad'. If 'ali' or 'alaa' is present, target_name = 'std_ali'. If 'salma' or 'hanan', target_name = 'std_salma'. For all other student names, it unconditionally evaluates Ziad.

Why It Is a Problem: The agent completely ignores the user's requested student if they are not in the 3 hardcoded names. Any query for 'Mohamed', 'Omar', 'Karim', or other members returns Ziad's scores. Furthermore, 'hanan' is incorrectly mapped to 'std_salma'.

Expected Behavior: The agent should dynamically extract the student name or student code from the query text, resolve it via IdentityMatcher / tool_get_student against the database, and fail gracefully if not found.

Concrete Attack / Failure Scenario: An HR leader asks the agent: 'Show evaluation score for Mohamed Ibrahim'. Because 'ali' and 'salma' are not in the query, target_name defaults to 'std_ziad'. The agent replies with Ziad's score data while labeling it as the answer to Mohamed's query, misleading HR leadership.

Recommended Fix Direction (High-Level): Replace hardcoded if/elif chains with an entity extraction routine that passes the parsed name to tool_get_student for dynamic database resolution.

[ISSUE-03] Automated Background Engine Bypasses Human-in-the-Loop WhatsApp Guardrails

Severity: **CRITICAL** | **Category:** Workflow / Human-in-the-Loop | **Status:** **CONFIRMED BUG / DESIGN ISSUE**

File: backend/app/services/automation_service.py

Component / Endpoint: AutomationEngine.run_attendance_cycle & run_task_cycle (lines 168-185, 324-340)

Current Code Behavior: The background scheduler periodic task scans ended meetings and overdue tasks. When unexcused absentees or overdue submitters are detected, it invokes openwa.send_message directly, automatically sending WhatsApp messages to students.

Why It Is a Problem: Directly violates the core architectural rule: 'The system flag should NOT automatically send WhatsApp messages unless explicitly authorized. The HR Member should remain in the human-in-the-loop process.'

Expected Behavior: The automation cycle should generate MemberFollowupStatus flags in PENDING state and notify the assigned HR Member, who then initiates contact via the WhatsApp chat window or wa.me deep link.

Concrete Attack / Failure Scenario: A meeting concludes and attendance is processed. A student had an accepted emergency excuse that was not yet entered into the system. The background engine immediately fires an automated official WhatsApp reprimand to the student without any HR member reviewing or confirming the dispatch.

Recommended Fix Direction (High-Level): Remove automated outbound WhatsApp dispatches from run_attendance_cycle and run_task_cycle. Restrict automation cycles to creating and updating MemberFollowupStatus flags for HR action.

[ISSUE-04] Missing Role & Account Status Verification on Real-time WhatsApp WebSocket

Severity: **CRITICAL** | **Category:** Security / WebSocket Authorization | **Status:** **SECURITY RISK / CONFIRMED BUG**

File: backend/app/api/routes_whatsapp.py

Component / Endpoint: whatsapp_chat_websocket (lines 498-520)

Current Code Behavior: The WebSocket endpoint /api/whatsapp/ws validates JWT signature and extracts user_id, but never queries the database to verify if user.is_active is True or if user.role is an authorized HR role.

Why It Is a Problem: A deactivated user or a non-HR committee member (role='committee_member') can open a WebSocket connection and receive real-time push events, potentially intercepting live messages.

Expected Behavior: WebSocket handshake must verify that the user exists, is_active is True, and the user possesses an HR role (committee_hr_member, committee_hr_leader, region_hr_head, hr_admin).

Concrete Attack / Failure Scenario: A committee member logs into the platform, obtains a standard member JWT, and connects to wss://.../api/whatsapp/ws?token=<token>. The connection is accepted. If an unassigned student message arrives, broadcast notifications are pushed to all connected users.

Recommended Fix Direction (High-Level): Enforce database-backed user validation during the WebSocket handshake before calling ws_manager.connect, terminating unauthorized roles with code 1008 (Policy Violation).

4. High Severity Issues

[ISSUE-05] Social Media HR Member Blocked by HTTP 403 from Collecting Task Scores

Severity: **HIGH** | **Category:** Workflow / Role Permission Boundary | **Status:** **CONFIRMED BUG**

File: backend/app/api/routes_tasks.py

Component / Endpoint: list_submissions_for_task & get_submission (lines 253-257, 310-314)

Current Code Behavior: In list_submissions_for_task and get_submission: 'if current_user.role in ("region_hr_head", "committee_hr_leader", "committee_hr_member"): raise HTTPException(403, detail="Access forbidden: HR roles do not have permission to view technical task submissions.")'

Why It Is a Problem: The project workflow mandates: 'Social Media HR Member: Collects task scores from Committee Head.' Under current code, all HR roles are blocked from reading task scores or submissions, making score collection impossible.

Expected Behavior: HR Members should have read access to the numerical task score and grading status for students in their committee, while potentially redacting proprietary technical submission files.

Concrete Attack / Failure Scenario: Committee Head grades Task #1 submissions. The Social Media HR Member logs in to compile the monthly scorecard. When calling /api/tasks/{task_id}/submissions, the API returns HTTP 403 Forbidden. The HR member cannot collect or record the scores.

Recommended Fix Direction (High-Level): Adjust the permission boundary to permit HR roles read-only access to score and status metadata on submissions, while redacting deliverable file URLs.

[ISSUE-06] Insecure Direct Object Reference (IDOR) in Feedback Review Status Endpoint

Severity: **HIGH** | **Category:** Security / Object-Level Authorization | **Status:** **SECURITY RISK / CONFIRMED BUG**

File: backend/app/api/routes_feedback.py

Component / Endpoint: update_feedback_status (lines 168-189)

Current Code Behavior: Endpoint PATCH /api/feedback/{feedback_id}/status allows role committee_hr_leader to update status. It fetches MemberFeedback by ID and commits changes without verifying that the feedback's student belongs to the caller's committee team_id.

Why It Is a Problem: An HR Leader from Committee A can review, modify, and action confidential feedback originating from Committee B simply by guessing or providing the feedback UUID.

Expected Behavior: The system must join Student on feedback.student_id and verify student.team_id == current_user.team_id before allowing modification.

Concrete Attack / Failure Scenario: HR Leader of Tech Committee submits a PATCH request to /api/feedback/fb_media_12345/status with status='ACTIONED'. The server commits the change, allowing cross-committee tampering with confidential HR feedback.

Recommended Fix Direction (High-Level): Add an explicit committee scope check ensuring Student.team_id == current_user.team_id before committing status updates.

[ISSUE-07] Insecure Direct Object Reference (IDOR) in Question Answering Endpoint

Severity: **HIGH** | **Category:** Security / Object-Level Authorization | **Status:** SECURITY RISK / CONFIRMED BUG

File: backend/app/api/routes_questions.py

Component / Endpoint: answer_question (lines 119-140)

Current Code Behavior: Endpoint POST /api/questions/{question_id}/answer allows role committee_head to post an answer. It retrieves MemberQuestion by ID and updates answer without checking if question.team_id == current_user.team_id.

Why It Is a Problem: Committee Heads can overwrite or answer questions directed to other committee heads, violating committee isolation.

Expected Behavior: Only the Committee Head whose team_id matches question.team_id should be authorized to answer.

Concrete Attack / Failure Scenario: Social Media Committee Head sends an answer to a question submitted to the Technical Committee (q.team_id == 'team_tech'). The endpoint accepts the request and sets answered_by_user_id to the Social Media Head.

Recommended Fix Direction (High-Level): Add check: if q.team_id != current_user.team_id: raise HTTPException(403, detail='Cannot answer questions for another committee.').

[ISSUE-08] Unlinked Users Impersonate First Database Student in Feedback & Question Creation

Severity: **HIGH** | **Category:** Security / Data Integrity | **Status:** CONFIRMED BUG

File: backend/app/api/routes_feedback.py & routes_questions.py

Component / Endpoint: submit_feedback (lines 45-49) & ask_question (lines 36-38)

Current Code Behavior: If current_user has no student_id and their email does not match a student, the code executes: 'std_first = await db.execute(select(Student.id)); student_id = std_first.scalars().first()'.

Why It Is a Problem: Arbitrary non-student accounts (such as administrators or HR members testing the endpoint) silently submit questions or confidential feedback under the identity of the very first student in the database.

Expected Behavior: If the authenticated user cannot be resolved to a valid student profile, the request must fail with HTTP 400 or 403.

Concrete Attack / Failure Scenario: An admin tests the question submission endpoint while not linked to a student. The question is stored in the database as having been asked by Ziad Mohamed (the first seeded student), corrupting Ziad's audit trail.

Recommended Fix Direction (High-Level): Remove the fallback to std_first. If student_id cannot be determined from current_user, return HTTP 400 Bad Request immediately.

[ISSUE-09] SQLite Foreign Key Enforcement Disabled by Default (Missing PRAGMA)

Severity: **HIGH** | **Category:** Database / Data Integrity | **Status:** DATABASE FINDING / CONFIRMED BUG

File: backend/app/core/database.py

Component / Endpoint: engine initialization (lines 43-45)

Current Code Behavior: In database.py, SQLite connections configure check_same_thread=False, but do not attach an event listener to execute 'PRAGMA foreign_keys=ON'.

Why It Is a Problem: SQLite disables foreign key constraint checking by default. Invalid foreign keys (e.g., non-existent student_id, team_id, or user_id) can be inserted without error, resulting in orphaned records.

Expected Behavior: Every SQLite connection must execute 'PRAGMA foreign_keys=ON' upon establishment.

Concrete Attack / Failure Scenario: A record is inserted into member_followup_statuses with hr_member_id='usr_non_existent'. SQLite accepts the row without complaint, resulting in foreign key corruption and JOIN failures.

Recommended Fix Direction (High-Level): Add an event listener to the SQLite engine:
`@event.listens_for(engine.sync_engine, 'connect') def set_sqlite_pragma(dbapi_connection, ...): cursor = dbapi_connection.cursor(); cursor.execute('PRAGMA foreign_keys=ON'); cursor.close().`

[ISSUE-10] Circular Foreign Key Creation Order in Alembic Baseline Migration

Severity: HIGH | **Category:** Database / Migration | **Status:** DATABASE FINDING / CONFIRMED BUG

File: backend/alembic/versions/3f35fadc1e13_baseline_schema.py

Component / Endpoint: upgrade() (lines 56-104)

Current Code Behavior: Alembic creates table 'students' at line 56 with foreign keys referencing 'users.id' and 'teams.id'. However, table 'teams' is created at line 79, and table 'users' is created at line 90.

Why It Is a Problem: While SQLite tolerates creating foreign keys to non-existent tables when foreign keys are disabled, executing this migration on PostgreSQL (Supabase) fails immediately with relation 'users' does not exist.

Expected Behavior: Tables must be created in dependency order (teams -> users -> students) or foreign key constraints must be added via separate alter table commands.

Concrete Attack / Failure Scenario: A developer provisions a cloud Supabase PostgreSQL database and runs 'uv run alembic upgrade head'. The migration crashes on op.create_table('students') because referenced table 'users' does not exist.

Recommended Fix Direction (High-Level): Reorder table creation in the baseline migration: create teams, then users, then students, and add cross-referencing foreign keys subsequently.

[ISSUE-11] Unauthenticated WhatsApp Webhook Ingestion in Non-Production Environments

Severity: HIGH | **Category:** Security / Webhook Validation | **Status:** SECURITY RISK

File: backend/app/api/routes_whatsapp.py

Component / Endpoint: openwa_webhook (lines 477-488)

Current Code Behavior: If OPENWA_WEBHOOK_SECRET is not configured and ENVIRONMENT != 'production', the webhook endpoint allows unauthenticated POST requests with any payload.

Why It Is a Problem: Attackers or unauthorized clients on the network can POST forged incoming student messages, spoofing student-to-HR conversations and injecting malicious text or media URLs into the chat database.

Expected Behavior: Webhook ingestion should require a secret key or shared HMAC signature across all environments.

Concrete Attack / Failure Scenario: An attacker sends a POST request to `/api/whatsapp/webhook` with a crafted JSON payload claiming to be from student phone '+201012345678'. The backend accepts the message and pushes it to the HR Member's chat UI.

Recommended Fix Direction (High-Level): Enforce webhook secret verification unconditionally, or require API key authentication even in development environments.

[ISSUE-12] Shared Single-Session OpenWA Daemon (Lack of Multi-HR Session Isolation)

Severity: **HIGH** | **Category:** Architecture / WhatsApp Integration | **Status:** DESIGN/ARCHITECTURE ISSUE

File: `backend/app/providers/openwa_provider.py` & `routes_whatsapp.py`

Component / Endpoint: `OpenWAProvider` (singleton) & `send_outgoing_message` (line 278)

Current Code Behavior: `OpenWAProvider` is instantiated as a global singleton referencing `settings.OPENWA_SESSION_ID` and `settings.OPENWA_OFFICIAL_PHONE`. When HR members send messages via the web UI, all messages originate from this single phone number.

Why It Is a Problem: There is no personal WhatsApp session isolation per HR member at the provider layer. All outbound messages appear to students as coming from the organization's official bot number rather than the designated HR member.

Expected Behavior: The architecture should either support multi-session OpenWA instances (one per HR member) or explicitly mandate that official broadcasts use OpenWA while personal HR follow-up exclusively uses wa.me deep links.

Concrete Attack / Failure Scenario: HR Member A and HR Member B both chat with their respective students simultaneously through the web window. Both students receive messages from the exact same phone number, creating confusion about who is contacting them.

Recommended Fix Direction (High-Level): Clarify the architectural boundary: use OpenWA exclusively for automated notifications and wa.me deep links for personal HR follow-ups, or implement multi-tenant session routing in `OpenWAProvider`.

5. Medium & Low Severity Issues

[ISSUE-13] Database Mutation Side Effects in Read-Only GET Meeting Attendance

Severity: **MEDIUM** | Category: API Design | Status: **CONFIRMED BUG**

File: backend/app/api/routes_attendance.py

Component / Endpoint: get_meeting_attendance (lines 172-180)

Current Code Behavior: In GET /attendance/meetings/{meeting_id}, if no records exist, it executes await attendance_service.process_meeting_attendance(meeting.id, db), creating records and generating absenteeism flags.

Why It Is a Problem: Violates HTTP GET idempotency and safety principles. Read requests should never mutate database state or create follow-up flags.

Expected Behavior: GET requests must be strictly read-only. Attendance processing must only occur on POST /attendance/meetings/{meeting_id}/process.

Concrete Attack / Failure Scenario: A search crawler or dashboard pre-fetcher sends a GET request to a meeting URL. Absenteeism flags are prematurely created before the meeting has even occurred.

Recommended Fix Direction (High-Level): Remove auto-processing from the GET endpoint; return empty records list if not yet processed.

[ISSUE-14] Missing API Endpoints for Manual Attendance Recording and Excuse Management

Severity: **MEDIUM** | Category: Workflow / Attendance | Status: **MISSING IMPLEMENTATION**

File: backend/app/api/routes_attendance.py

Component / Endpoint: Module level

Current Code Behavior: The attendance module only supports Google Meet automated log processing. There are no REST endpoints for an HR Member to manually record or edit attendance or submit excuses.

Why It Is a Problem: HR members cannot manually override a member's attendance status or submit excused absence notes.

Expected Behavior: HR members should be able to call PUT/POST /attendance/records/{id} to set status to EXCUSED_ACCEPTED or PRESENT.

Concrete Attack / Failure Scenario: A student informs HR in advance of an exam. The HR member has no API endpoint to record the excuse, resulting in the student being marked UNEXCUSED_ABSENT.

Recommended Fix Direction (High-Level): Implement CRUD endpoints for AttendanceRecord allowing HR members to update status and excuse notes.

[ISSUE-15] Absenteeism Follow-Up Assigned to Committee Head Instead of HR Member

Severity: **MEDIUM** | Category: Workflow / Attendance Policy | Status: **CONFIRMED BUG**

File: backend/app/services/attendance_service.py

Component / Endpoint: process_meeting_attendance (lines 218-228)

Current Code Behavior: When a student is marked UNEXCUSED_ABSENT, line 220 assigns: `hr_id = student.assigned_hr_id` or `meeting.responsible_user_id` or `'usr_hr_member'`.

Why It Is a Problem: If student has no assigned HR, the flag is assigned to `meeting.responsible_user_id` (the Committee Head who scheduled the meeting), even though Committee Heads do not perform HR WhatsApp follow-ups.

Expected Behavior: Unassigned student flags should be assigned to the Committee HR Leader (`committee_hr_leader`) for delegation.

Concrete Attack / Failure Scenario: A meeting has absent students without assigned HRs. Follow-up tasks appear in the Committee Head's queue, who lacks HR WhatsApp tools to handle them.

Recommended Fix Direction (High-Level): Fallback to the committee's HR Leader rather than the meeting's `responsible_user_id`.

[ISSUE-16] Pre-Meeting Automated Reminders Completely Missing from Scheduling Engine

Severity: MEDIUM | **Category:** Workflow / Automation | **Status:** MISSING IMPLEMENTATION

File: backend/app/services/automation_service.py

Component / Endpoint: run_attendance_cycle (lines 108-126)

Current Code Behavior: `run_attendance_cycle` only queries meetings where `start_time <= now`. There is zero logic in the background scheduler to send reminders before an upcoming meeting.

Why It Is a Problem: The documented workflow 'Before meeting: System -> reminder to member' is unimplemented in the background automation cycle.

Expected Behavior: The scheduler should scan upcoming meetings (e.g. starting in 24 hours or 1 hour) and dispatch pre-meeting reminders.

Concrete Attack / Failure Scenario: A meeting is scheduled for tomorrow at 6:00 PM. No automated reminder is ever dispatched by the background automation engine.

Recommended Fix Direction (High-Level): Add a `run_meeting_pre_cycle` method to `AutomationEngine` that scans meetings starting within the configured reminder window.

[ISSUE-17] Missing 'Interaction /5' Score Component in Domain Models and Services

Severity: MEDIUM | **Category:** Scoring System | **Status:** MISSING IMPLEMENTATION

File: backend/app/models/entities.py & schemas.py

Component / Endpoint: BehaviorScoreUpdate, ScoringService

Current Code Behavior: Models and services implement Behavior /23 (4 sub-categories totaling 23) and Bonus points, but completely omit the distinct 'Interaction /5' component.

Why It Is a Problem: The system cannot store or evaluate the standalone Interaction score required by the organization's HR specification.

Expected Behavior: ScoreRecord and schemas should support an explicit 'INTERACTION' category scored out of 5.0.

Concrete Attack / Failure Scenario: An HR member attempts to enter an Interaction score out of 5 for a student; the backend has no field or category for it.

Recommended Fix Direction (High-Level): Extend ScoreRecord categories and BehaviorScoreUpdate schema to include interaction: float = Field(..., ge=0.0, le=5.0).

[ISSUE-18] Global Organization Data Leakage in Committee Report Metrics Summary

Severity: MEDIUM | **Category:** Security / Data Isolation | **Status:** SECURITY RISK / CONFIRMED BUG

File: backend/app/api/routes_reports.py

Component / Endpoint: get_committee_summary (lines 77-93)

Current Code Behavior: In get_committee_summary, open_flags, feedback_count, and total_bonuses count records across the entire database without filtering by Student.team_id == team_id.

Why It Is a Problem: Leaks organization-wide operational statistics into an individual committee's summary report.

Expected Behavior: All aggregations in a committee summary must be strictly filtered by team_id.

Concrete Attack / Failure Scenario: Social Media HR Leader submits a report to the Regional Head; the report contains open flag and feedback counts from the Tech and Logistics committees.

Recommended Fix Direction (High-Level): Join Student on team_id in all count queries within get_committee_summary.

[ISSUE-19] ReAct Agent Tools Bound to Mock Providers Instead of Production OpenWA

Severity: MEDIUM | **Category:** AI Agent / Tools | **Status:** DESIGN/ARCHITECTURE ISSUE

File: backend/app/agent/tools.py

Component / Endpoint: tools.py initializers (lines 28-34, 472)

Current Code Behavior: tools.py instantiates mock_meet_provider, mock_cal_provider, and mock_msg_provider. tool_send_reminder dispatches reminders via MockMessagingProvider.

Why It Is a Problem: Reminders confirmed through the agent chat interface are only recorded in-memory by the mock provider and never sent to real WhatsApp recipients.

Expected Behavior: Agent tools should use production OpenWAProvider and Google Meet providers.

Concrete Attack / Failure Scenario: HR Leader confirms a reminder action in /agent/confirm. The action returns success, but no WhatsApp message is ever transmitted via OpenWA.

Recommended Fix Direction (High-Level): Inject the live OpenWAProvider into reminder_service inside tools.py.

[ISSUE-20] Non-Deterministic First-Match Bias in Student Identity Matcher

Severity: MEDIUM | **Category:** Identity Matching | **Status:** CONFIRMED BUG

File: backend/app/services/identity_matcher.py

Component / Endpoint: match_participant (lines 90-104)

Current Code Behavior: IdentityMatcher iterates through students in arbitrary database order and returns the first student matching ≥ 2 tokens immediately.

Why It Is a Problem: If two students share similar names (e.g. 'Ahmed Mohamed Ali' and 'Ahmed Mohamed Hassan'), whichever student appears first in the DB is matched, and the other is ignored.

Expected Behavior: IdentityMatcher should evaluate all candidates, detect ambiguous ties, and flag them for human review if multiple students match with equal confidence.

Concrete Attack / Failure Scenario: Both Ahmed Ali and Ahmed Hassan attend a meeting. The log lists 'Ahmed Mohamed'. Both sessions match Ahmed Ali because he appears first in the DB query.

Recommended Fix Direction (High-Level): Collect all matching candidates with confidence scores; if top scores are tied, return MatchResult(confidence=0.5, matched_by='AMBIGUOUS').

[ISSUE-21] Naive Timezone Overwrite in Attendance Status Evaluation

Severity: MEDIUM | **Category:** Policy Engine | **Status:** CONFIRMED BUG

File: backend/app/services/attendance_service.py

Component / Endpoint: AttendancePolicyEngine.evaluate_status (lines 42-46)

Current Code Behavior: Uses .replace(tzinfo=timezone.utc) to force timezone awareness between meeting_start and first_join.

Why It Is a Problem: Overwriting tzinfo without conversion alters the underlying hour value if local time offsets (e.g. UTC+2 Egypt) are used, distorting join delay calculations.

Expected Behavior: Must use .astimezone(timezone.utc) for proper timezone conversion.

Concrete Attack / Failure Scenario: A meeting starts at 18:00 UTC (20:00 Cairo). A participant joins at 20:02 Cairo (naive). Replace forces it to 20:02 UTC, calculating a 122-minute delay instead of 2 minutes.

Recommended Fix Direction (High-Level): Use .astimezone(timezone.utc) or ensure all datetimes are stored and normalized as timezone-aware UTC.

[ISSUE-22] Hardcoded 'today_sync' Identifier in Dashboard Statistics Endpoint

Severity: LOW | **Category:** API Design | **Status:** CONFIRMED BUG

File: backend/app/api/routes_dashboard.py

Component / Endpoint: get_dashboard_stats (line 51)

Current Code Behavior: Dashboard queries AttendanceRecord.meeting_id == 'today_sync' directly.

Why It Is a Problem: If today's meeting code is not named 'today_sync', today's attendance stats will always display 0 / 100%.

Expected Behavior: The dashboard should query the most recent meeting matching today's date.

Concrete Attack / Failure Scenario: A meeting is scheduled with code 'sync_media_01'. Dashboard attendance cards show 0 attendees because the code does not equal 'today_sync'.

Recommended Fix Direction (High-Level): Query Meeting where `func.date(start_time) == func.date(utcnow())` instead of hardcoding `'today_sync'`.

[ISSUE-23] Timestamp Collision Risk in ReminderLog Primary Keys

Severity: LOW | **Category:** Database / Primary Key | **Status:** CONFIRMED BUG

File: backend/app/services/reminder_service.py

Component / Endpoint: send_reminders (line 104)

Current Code Behavior: Generates ID: `rem_{int(datetime.now().timestamp())}_{s.id}`.

Why It Is a Problem: If two reminders are dispatched to the same student within the same second, primary key collision occurs and aborts the transaction.

Expected Behavior: Primary keys should use UUIDs (e.g. `rem_{uuid4().hex[:12]}`).

Concrete Attack / Failure Scenario: A batch retry dispatches two reminders in the same second, crashing the transaction with `IntegrityError: UNIQUE constraint failed: reminder_logs.id`.

Recommended Fix Direction (High-Level): Use `uuid.uuid4().hex` to guarantee globally unique reminder log IDs.

[ISSUE-24] Unbounded In-Memory Conversational State Memory Leak

Severity: LOW | **Category:** AI Agent / Memory | **Status:** POTENTIAL ISSUE

File: backend/app/agent/react_agent.py

Component / Endpoint: CONVERSATION_STATE (line 18)

Current Code Behavior: `CONVERSATION_STATE` is a global Python dict that retains conversation history indefinitely.

Why It Is a Problem: In long-running production processes, unbounded dictionary growth will lead to gradual memory exhaustion (OOM).

Expected Behavior: Conversational state should have a TTL cache (e.g., `cachetools.TTLCache`) or be persisted in Redis/database.

Concrete Attack / Failure Scenario: After thousands of agent interactions over weeks, server memory usage steadily increases.

Recommended Fix Direction (High-Level): Implement an LRU or TTL cache mechanism with a maximum of 1,000 active conversations.

[ISSUE-25] Phone Suffix Collision Risk in OpenWA Webhook Ingestion

Severity: LOW | **Category:** WhatsApp / Webhook | **Status:** POTENTIAL ISSUE

File: backend/app/services/whatsapp_service.py

Component / Endpoint: handle_webhook_event (lines 446-456)

Current Code Behavior: Matches incoming phone numbers using `Student.phone.like(f'%{suffix}')` and takes the first match.

Why It Is a Problem: If two students share the same 9 digits across different country codes, incoming messages could be misattributed.

Expected Behavior: Must match on full normalized international E.164 phone numbers.

Concrete Attack / Failure Scenario: An international student shares 9 digits with an Egyptian domestic number. Messages from one are attributed to the other.

Recommended Fix Direction (High-Level): Require exact international phone number matching after standardizing both sender and student records to E.164 format.

6. Authentication & Authorization Findings

Role-Based Access Control (RBAC):

The backend defines 5 authoritative operational roles in UserRole: region_hr_head, committee_hr_leader, committee_head, committee_hr_member, and committee_member. The dependency require_roles correctly expands legacy aliases (such as team_lead and member).

Vulnerabilities Identified:

- **Pre-Auth Student Account Hijacking (ISSUE-01):** Registration binds student identities without email verification.
- **Regional HR Head Over-Privilege:** verify_student_access unconditionally grants region_hr_head access across all modes ('read', 'write', 'chat'). This contradicts the organizational workflow where Regional Head has high-level oversight and does not engage in operational student chats.
- **HR Member Score Access Block (ISSUE-05):** Strict 403 blocks prevent HR Members from reading or collecting task scores.
- **IDOR in Feedback & Question Endpoints (ISSUE-06, ISSUE-07):** Status updates and question answers fail to check committee scope.

7. Database Findings

Model Definitions & SQLite Realities:

- **Foreign Key Enforcement (ISSUE-09):** SQLite does not enforce foreign keys without explicit PRAGMA foreign_keys=ON. Because this pragma is not executed on connection, SQLite ignores all ForeignKey definitions.
- **Migration Sequencing (ISSUE-10):** Alembic baseline migration 3f35fadc1e13_baseline_schema.py attempts to create table students referencing users before users exists, failing on PostgreSQL.
- **Unique Constraints & Indices:** Tables meeting_assignments, task_assignments, and score_records correctly specify compound unique constraints, preventing duplicate assignments.
- **Async Database Usage:** SQLAlchemy AsyncSession is consistently used with asyncpg/aiosqlite; no blocking sync DB calls were detected.

8. API Findings

- **GET Side Effects (ISSUE-13):** GET /attendance/meetings/{meeting_id} triggers attendance evaluation and DB writes on read.
- **Dashboard Hardcoding (ISSUE-22):** GET /dashboard/stats filters by hardcoded today_sync.
- **Cross-Tenant Metrics Leakage (ISSUE-18):** GET /reports/committee/summary leaks organization-wide flag and feedback counts.
- **Impersonation in Form Submissions (ISSUE-08):** submit_feedback and ask_question default to the first student in the DB for unlinked users.

9. AI Agent Findings

- **Hardcoded Demo Identities (ISSUE-02):** Evaluation queries default to Ziad Mohamed regardless of the requested member name.
- **Mock Provider Execution (ISSUE-19):** Agent tools in tools.py execute against in-memory mock providers rather than live OpenWA.
- **Hardcoded Meeting IDs:** Attendance queries in react_agent.py are hardcoded to today_sync.
- **In-Memory State Leak (ISSUE-24):** Global dictionary CONVERSATION_STATE grows unbounded with no eviction.

10. WhatsApp & OpenWA Findings

- **Bypass of Human-in-the-Loop (ISSUE-03):** Background automation directly dispatches WhatsApp messages on meeting absence and overdue tasks.
- **WebSocket Authorization Flaw (ISSUE-04):** /api/whatsapp/ws does not verify role or active status.
- **Shared Single Session (ISSUE-12):** All HR members share one official OpenWA session and phone number.
- **Unauthenticated Webhooks in Dev (ISSUE-11):** /api/whatsapp/webhook accepts unauthenticated POSTs when secret is omitted in non-prod.

11. Workflow Compliance Findings

Workflow Area	Expected Workflow	Compliance Status & Gaps
A) Task Workflow	Head creates -> Member submits -> Head scores -> HR collects score	NON-COMPLIANT: HR roles blocked by 403 from viewing/collecting task scores (ISSUE-05).
B) Meeting / Attendance	Head schedules -> Member attends -> HR records attendance	PARTIALLY COMPLIANT: Attendance derived from Meet logs only; manual recording and excuse endpoints missing (ISSUE-14). Absenteeism flags misrouted to Head (ISSUE-15).
C) Automated Reminders	Pre-meeting reminder & pre-deadline reminder to member	PARTIALLY COMPLIANT: Pre-meeting reminders missing (ISSUE-16). Automated post-deadline messages bypass HITL (ISSUE-03).
D) System Flag -> HITL WhatsApp	Flag created -> HR member opens chat window -> Sends to member	NON-COMPLIANT: Automation engine sends directly without HR review (ISSUE-03). All HRs share single number (ISSUE-12).
E) Scoring System	Task /10, Behavior /23, Interaction /5, Bonus points. No fake total formula.	MOSTLY COMPLIANT: Behavior /23 and Bonus points stored correctly. Formula is NOT invented. 'Interaction /5' component is missing (ISSUE-17).
F) Member Feedback	Member -> Feedback -> HR Leader actions. Confidential from HR Member & Head.	PARTIALLY COMPLIANT: Confidentiality from HR members enforced, but IDOR in status update (ISSUE-06) and unlinked user impersonation (ISSUE-08) present.
G) Member Questions	Member -> Question -> Committee Head answers	PARTIALLY COMPLIANT: Basic flow works, but IDOR in answering across committees (ISSUE-07) and first student impersonation (ISSUE-08) present.

12. Recommended Fix Priority

Phase 1: Critical Security & Integrity Remediation (Immediate)

1. Fix routes_auth.py registration to eliminate pre-auth student account takeover (ISSUE-01).
2. Add active and role checks to the WhatsApp WebSocket handshake (ISSUE-04).
3. Disable direct WhatsApp dispatch in automation_service.py to restore human-in-the-loop guardrails (ISSUE-03).
4. Fix react_agent.py to dynamically resolve student names instead of hardcoding demo IDs (ISSUE-02).

Phase 2: Authorization & Workflow Alignment (High Priority)

5. Update routes_tasks.py to allow HR roles read-only access to task scores (ISSUE-05).
6. Add committee-scope verification in feedback status (ISSUE-06) and question answer (ISSUE-07) endpoints.
7. Remove student impersonation fallbacks in feedback and question submission endpoints (ISSUE-08).
8. Enable SQLite PRAGMA foreign_keys=ON in database connection configuration (ISSUE-09).
9. Fix table creation order in Alembic baseline migration (ISSUE-10).

Phase 3: Operational Completeness & Quality (Medium/Low Priority)

10. Add manual attendance and excuse management endpoints (ISSUE-14).
11. Implement pre-meeting reminder evaluation in the background scheduler (ISSUE-16).
12. Add the missing 'Interaction /5' score component to entities and schemas (ISSUE-17).
13. Scope committee summary metrics by team_id (ISSUE-18).
14. Remove mutation side-effects from GET /attendance/meetings/{meeting_id} (ISSUE-13).

13. Final Conclusion

The StudentOps AI backend possesses a strong architectural foundation, clean async patterns, and rigorous deterministic policy engines. However, operational compliance with the multi-role student organization workflow is hindered by access control mismatches (HR blocked from task scores, IDOR vulnerabilities across committees), automated message dispatching that bypasses the human-in-the-loop requirement, and demo-specific hardcodings in the AI agent. Resolving the 25 concrete findings documented in this report will bring the backend to production-grade security, deterministic reliability, and complete workflow fidelity.